

PREDICTIVE MAINTENANCE: DATASET, APPROACH, AND RESULTS

1. DATASET

The file `predictive_maintenance.csv` has 10,000 machine-operation records and 10 columns. Each row is one reading from one machine, and the rows are in sequential order by UDI.

Column	Type	Role in the model
UDI	integer	Row index. Dropped from features; used only for the chronological split
Product ID	text	Identifier. Dropped
Type	category (L, M, H)	Feature, one-hot encoded
Air temperature [K]	numeric	Feature
Process temperature [K]	numeric	Feature
Rotational speed [rpm]	numeric	Feature
Torque [Nm]	numeric	Feature
Tool wear [min]	numeric	Feature
Target	binary	Label to predict (1 = failure)
Failure Type	category	Excluded. It is a post-failure diagnosis, so it would leak the answer

- No missing values and no duplicate rows. No imputation was needed, though the pipeline includes it as a safeguard.
- Failures are rare. There are 339 failures (3.4%), a ratio of about 28 non-failures to every failure.
- Machine types are unevenly represented. L has 6,000 rows, M has 2,997, and H has 1,003. Failure rates are 3.9%, 2.8%, and 2.1% respectively.
- Failures are concentrated in certain operating conditions. Failure rates rise sharply at low rotational speed and high torque, and grow with tool wear. The linear correlations are weak, so the relationships are non-linear.
- Failures cluster in time. The UDI block covering roughly rows 4,001 to 5,000 contains 134 of the 339 failures, about 40%. This suggests a regime change in that period.
- 27 rows have conflicting labels. Nine have `Target = 1` with `Failure Type = No Failure`, and 18 have `Target = 0` with `Failure Type = Random Failures`. This is 0.27% of the data. `Target` is treated as the ground truth, and the conflicting rows are kept for the binary model and dropped from the multiclass model.

2. APPROACH

Feature engineering

Six engineered features are added in `add_features`. Each is computed row by row, so it cannot leak information across rows:

- $\text{Temp_diff} = \text{process temperature} - \text{air temperature}$ (heat dissipation)
- $\text{Power} = \text{torque} \times \text{rotational speed}$ (mechanical load)
- $\text{Torque_per_rpm} = \text{torque} \div \text{speed}$
- $\text{Wear_torque} = \text{tool wear} \times \text{torque}$ (worn tools under high load)
- $\text{Wear_rpm} = \text{tool wear} \times \text{speed}$
- $\text{Temp_ratio} = \text{process temperature} \div \text{air temperature}$

The final model uses 11 numeric features plus one-hot encoded `Type`.

Validation design

Two splits are used:

1. Stratified random split (80/20). This keeps the failure rate the same in both halves. It tests how well the model handles new machines drawn from the same mix as the training data. The test set has 2,000 rows and 68 failures.
2. Chronological split. Train on the first 80% of rows by UDI, test on the last 20%. The test window has 39 failures. This mimics deployment, where the model only sees the past, so it is the more realistic estimate.

Hyperparameters are tuned with `RandomizedSearchCV` (10 iterations, 5-fold stratified CV), optimising average precision (PR-AUC).

Handling imbalance

Three levers are used:

- Class weighting for models that support it (logistic regression, decision tree, random forest, extra trees, XGBoost via `scale_pos_weight`, CatBoost, SVM, LightGBM when available). Gradient boosting has no class weighting.
- Resampling was compared on a gradient boosting probe: none, SMOTE, ADASYN, random over- and under-sampling, SMOTE-ENN, and SMOTE-Tomek. Plain training (`none`) scored best (CV PR-AUC 0.914). Undersampling was clearly worst (0.723), since it discards most of the 271 training failures. The chosen resampler is applied to every model.
- Threshold selection. The decision threshold is chosen to minimise expected cost, with a missed failure costing 10 and a false alarm costing 1. These weights are assumptions, not measured costs, so change them to match your plant's economics.

Avoiding leakage

- All preprocessing sits inside the pipeline and is fitted only on training folds.
- The threshold is chosen from out-of-fold predictions on the training data only. The test set is used once, at the end.
- Models are ranked by CV PR-AUC on training data, not by test performance.

3. RESULTS

Model comparison (stratified test set, threshold from training data)

Model	CV PR-AUC	Test PR-AUC	Precision	Recall	Threshold	Expected cost
GB	0.914	0.905	0.950	0.838	0.28	113
RF	0.888	0.882	0.918	0.824	0.28	125
CatBoost	0.876	0.867	0.615	0.868	0.15	127
XGB	0.874	0.885	0.600	0.838	0.24	148

Best model: gradient boosting (200 trees, depth 3, learning rate 0.03)

On the 2,000-row test set:

- Confusion matrix: 1,929 correct non-failures, 3 false alarms, 57 caught failures, 11 missed failures
- ROC-AUC 0.974, Brier score 0.0066 (well calibrated)
- F1 0.891, MCC 0.889

The 11 missed failures are the main cost, giving an expected cost of $10 \times 11 + 3 = 113$.

Chronological check (same model)

- Threshold 0.05, PR-AUC 0.821, recall 0.795, precision 0.517, cost 109.
- Recall drops from 0.84 to 0.80, and precision drops to about 0.52. The later period has a different failure pattern, so this is the main caveat for deployment.

Multiclass: failure type (secondary)

A separate LightGBM model with balanced class weights predicts the failure type for the 9,800 rows that remain after removing conflicts. Macro-F1 is 0.75. Heat Dissipation, Overstrain, and Power failures are classified well. Tool Wear Failure is never identified (0 of 9 test cases). This is a feature limitation rather than a model error, so it should be reported as one.

4. SAVED ARTIFACT AND INFERENCE

`best_predictive_maintenance_model.joblib` contains the fitted pipeline, the threshold (0.28), the feature lists, the resampler, the cost weights, and the metrics. The copy you uploaded is byte-identical to the earlier one.

The `predict_failure` function validates inputs (type must be L, M, or H; numeric fields must be finite; speed must be positive), applies the same feature engineering, and returns a probability and a flag. In the example, a healthy-looking reading gets probability 0.0025 and is not flagged.

5. LIMITATIONS

- Small number of failures. The test set has only 68 failures and the chronological test has 39. A difference of about 0.02 in PR-AUC, such as between GB and XGB, is within noise.
- Threshold depends on costs. The 10:1 cost ratio is an assumption. Changing it moves the threshold and the trade-off between missed failures and false alarms.
- Time shift. Performance precision drops slightly by 5% on the later window.
- Tool Wear Failure is undetectable with the current features.
- Feature-importance caveat. Engineered features overlap with their raw inputs, so permutation importance can split credit between them. Rankings as indicative, not as proof that a raw feature is unimportant.

6. BOTTOM LINE

Gradient boosting is the best of the models tested. At threshold 0.28 it catches about 84% of failures with about 95% precision on unseen data, and it holds up reasonably on the time-ordered check. The main risks are the small number of failures, the cost assumptions, and the drop in recall on later data.